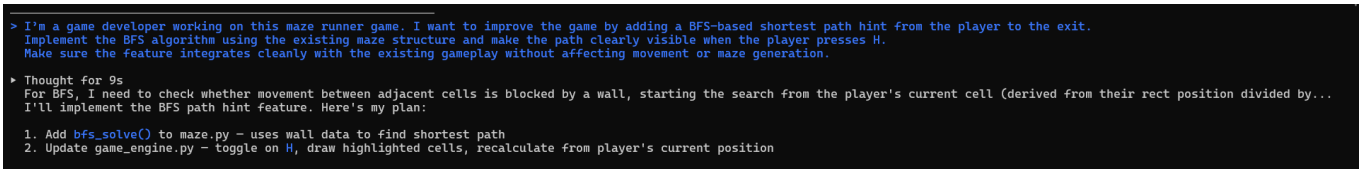
Maze Runner Software Engineering Lab 4

NAME - SATVIK DAS

SRN - PES2UG24CS448

GITHUB - https://github.com/satvik-2006/12_maze_runner

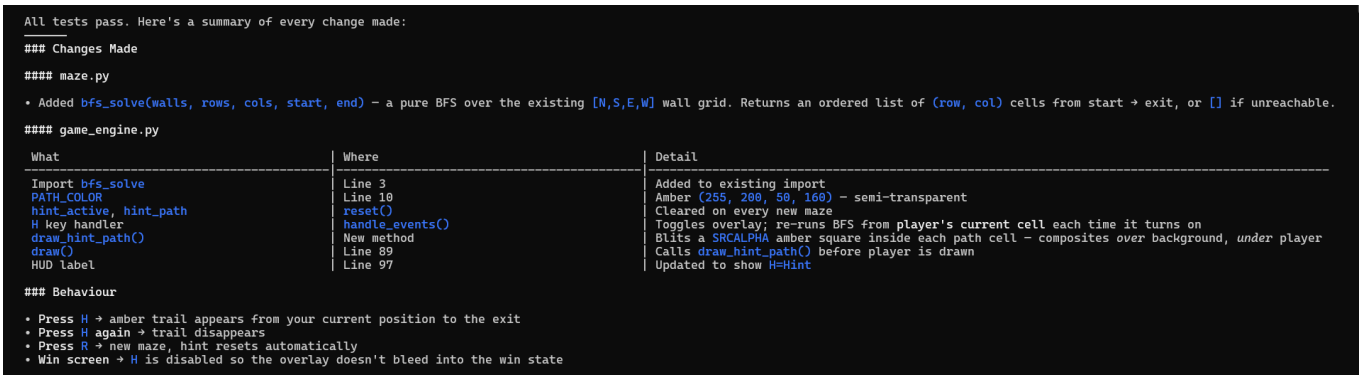
Task 1 - Shortest Path Hint and GitHub Commit



```
> I'm a game developer working on this maze runner game. I want to improve the game by adding a BFS-based shortest path hint from the player to the exit.
  Implement the BFS algorithm using the existing maze structure and make the path clearly visible when the player presses H.
  Make sure the feature integrates cleanly with the existing gameplay without affecting movement or maze generation.

▸ Thought for 9s
  For BFS, I need to check whether movement between adjacent cells is blocked by a wall, starting the search from the player's current cell (derived from their rect position divided by...
  I'll implement the BFS path hint feature. Here's my plan:

  1. Add bfs_solve() to maze.py — uses wall data to find shortest path
  2. Update game_engine.py — toggle on H, draw highlighted cells, recalculate from player's current position
```



All tests pass. Here's a summary of every change made:

Changes Made

maze.py

• Added `bfs_solve(walls, rows, cols, start, end)` — a pure BFS over the existing `[N,S,E,W]` wall grid. Returns an ordered list of `(row, col)` cells from start → exit, or `[]` if unreachable.

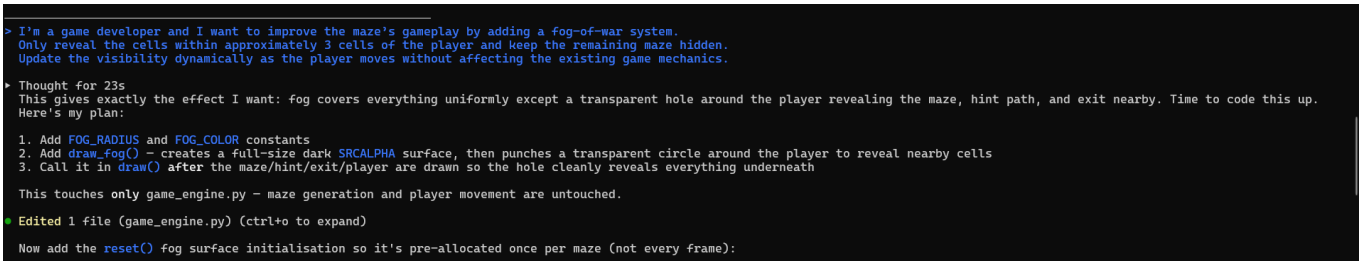
game_engine.py

What	Where	Detail
Import `bfs_solve`	Line 3	Added to existing import
`PATH_COLOR`	Line 10	Amber `(255, 200, 50, 160)` — semi-transparent
`hint_active`, `hint_path`	`reset()`	Cleared on every new maze
`H` key handler	`handle_events()`	Toggles overlay; re-runs BFS from **player's current cell** each time it turns on
`draw_hint_path()`	New method	Blits a `SRCALPHA` amber square inside each path cell — composites *over* background, *under* player
`draw()`	Line 89	Calls `draw_hint_path()` before player is drawn
HUD label	Line 97	Updated to show `H=Hint`

Behaviour

• **Press H** → amber trail appears from your current position to the exit
• **Press H again** → trail disappears
• **Press R** → new maze, hint resets automatically
• **Win screen** → H is disabled so the overlay doesn't bleed into the win state

Task 2 - Fog of War and Visibility Fix



```
> I'm a game developer and I want to improve the maze's gameplay by adding a fog-of-war system.
  Only reveal the cells within approximately 3 cells of the player and keep the remaining maze hidden.
  Update the visibility dynamically as the player moves without affecting the existing game mechanics.

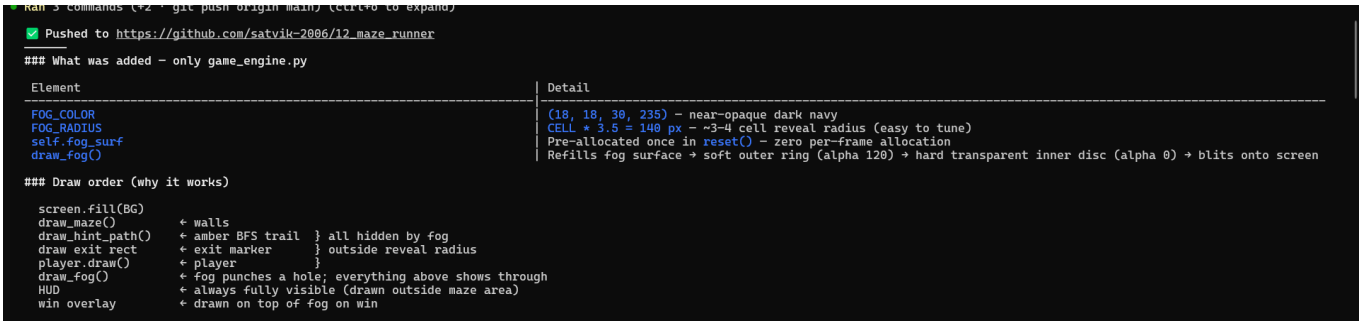
▸ Thought for 23s
  This gives exactly the effect I want: fog covers everything uniformly except a transparent hole around the player revealing the maze, hint path, and exit nearby. Time to code this up.
  Here's my plan:

  1. Add FOG_RADIUS and FOG_COLOR constants
  2. Add draw_fog() — creates a full-size dark SRCALPHA surface, then punches a transparent circle around the player to reveal nearby cells
  3. Call it in draw() after the maze/hint/exit/player are drawn so the hole cleanly reveals everything underneath

  This touches only game_engine.py — maze generation and player movement are untouched.

● Edited 1 file (game_engine.py) (ctrl+o to expand)

  Now add the reset() fog surface initialisation so it's pre-allocated once per maze (not every frame):
```



```
● Ran 3 commands (ctrl+o to expand)

  ✅ Pushed to https://github.com/satvik-2006/12_maze_runner
```

What was added — only game_engine.py

Element	Detail
`FOG_COLOR`	`(18, 18, 30, 235)` — near-opaque dark navy
`FOG_RADIUS`	`CELL * 3.5 = 140 px` — ~3–4 cell reveal radius (easy to tune)
`self.fog_surf`	Pre-allocated once in `reset()` — zero per-frame allocation
`draw_fog()`	Refills fog surface → soft outer ring (alpha 120) → hard transparent inner disc (alpha 0) → blits onto screen

Draw order (why it works)

```
screen.fill(BG)
draw_maze()        ← walls
draw_hint_path()   ← amber BFS trail  } all hidden by fog
draw exit rect     ← exit marker      } outside reveal radius
player.draw()      ← player           }
draw_fog()         ← fog punches a hole; everything above shows through
HUD                ← always fully visible (drawn outside maze area)
win overlay        ← drawn on top of fog on win
```

Task 3 - Timer Leaderboard

```
> I'm a game developer and I want to add a persistent timer-based leaderboard to the maze game.
  Track the player's completion time, save it to leaderboard.json, and maintain the top 5 fastest times.
  Display the leaderboard when the player completes the maze and make sure the data persists between game sessions.

▸ Thought for 49s
  Good, that fits within 520px. I'll highlight the player's own entry in a distinct color like gold, and use a relative "leaderboard.json" path so it saves to the project root when run...
  Here's my plan:

  1. New game/leaderboard.py — load_times() + save_time() with JSON persistence
  2. game_engine.py — trigger save on win, add a small_font, replace the win screen with a full leaderboard display
```

✅ Pushed to https://github.com/satvik-2006/12_maze_runner

What was added

New: leaderboard.py

Function	What it does
load_times()	Reads leaderboard.json, returns sorted [float]. Safe against missing/corrupt files.
save_time(elapsed)	Appends → sorts → truncates to 5 → writes JSON → returns (rank, times). rank=None if time missed top 5.

Modified: game_engine.py

__init__	*	added small_font (monospace 19)
reset()	*	loads leaderboard from disk; clears rank + leaderboard_times
update()	*	calls save_time() exactly once at the win-transition moment
draw()	*	full leaderboard win screen:

```
      Solved in 18.3s!              ← green, big_font
         NEW BEST!                  ← gold / silver / bronze / grey badge
      ──────── TOP 5 ────────
   1.  18.30s  <--                  ← your row in green
   2.  21.50s                       ← gold/silver/bronze/dim colours
   3.  25.10s
   4.  30.00s
   5.  35.20s
      Press R for a new maze
```

Task 4 - Difficulty Tiers

```
> I'm a game developer and I want to add multiple difficulty levels to the maze game.
  Add Easy (10×8), Medium (15×13), and Hard (20×18) and allow the player to select one before starting.
  Make sure the selected size is used for maze generation while keeping all existing features functional.

▸ Thought for 1m 40s
  Now I need to update game_engine.py and leaderboard.py — for the latter I'll add a difficulty parameter defaulting to "Medium" so existing calls stay backwards compatible.
  Here's my complete plan:

  • leaderboard.py: Add difficulty param so each difficulty has its own persistent scores (leaderboard_easy.json, etc.)
  • game_engine.py: Full rewrite — add DIFFICULTIES dict, difficulty-select screen state, make COLS/ROWS/WIDTH/HEIGHT instance variables, wire ESC to go back to menu, adapt all existing
  features

  Let me write both files cleanly:
```

✅ Pushed to https://github.com/satvik-2006/12_maze_runner

What was built

Select screen (new state: "select")

```
┌──────────────────────────────────────┐
│            MAZE  RUNNER              │
│       Select a difficulty to begin   │
│                                      │
│  ┌──────┐  ┌──────┐  ┌──────┐        │
│  │ EASY │  │MEDIUM│  │ HARD │        │
│  │10 x 8│  │15 x 13│ │20 x 18│       │
│  └──────┘  └──────┘  └──────┘        │
│       ESC to return here from the game│
└──────────────────────────────────────┘
```

- Buttons highlight on hover (brighter) and click to start
- Window **resizes** to match the chosen maze dimensions

State machine in game_engine.py

Key	Action
Click difficulty	Resize window → generate maze → start
R	New maze at same difficulty
H	BFS hint toggle (from current position)
ESC	Return to difficulty select

Leaderboard isolation in leaderboard.py

- leaderboard_easy.json / leaderboard_medium.json / leaderboard_hard.json
- Top-5 tracked **separately** per difficulty — a Hard best doesn't pollute Easy records
- Win screen header shows TOP 5 [HARD] etc. so the player knows which board they're on